Tracing the Consequences of Quantization in Closed-Loop Agents

Anonymous ACL submission

Abstract

When a weight-quantized LLM agent acts in a closed loop, a single changed action alters everything that follows, so aggregate task success and open-loop probability probes cannot say which consequences quantization causes or where they come from. We ask: on identical interaction histories, which task consequences do quantization-induced local action changes cause, and do they come from the current action, from the subsequent execution, or from their interaction? We build a same-state four-branch evaluation. At the first semantic action divergence between a BF16 agent and its W4 counterpart reading the identical history, we execute either agent's action and continue with either agent, which yields a one-replacement gain G, its split into rescues and new damage, and an additive decomposition of the total change into current-action, continuation and interaction terms. We run it on Qwen3-4B and Qwen3-8B with GPTQ-W4 and AWQ-W4 in τ^2-bench under byte-deterministic serving, with independent prompt-nonce blocks as replicates, blinded human review of the mechanism labels, and a frozen confirmation on a held-out telecom split. [PENDING: findings: one sentence per headline result, filled only from the development and confirmation analyses (§4.5, §6)]

1 Introduction

Weight quantization is how LLM agents are deployed, and its evaluation practice comes from single-turn benchmarks: compare the aggregate score of a quantized model with that of its full-precision reference. In a closed-loop agent benchmark such as τ^2-bench (Barres et al., 2025), the agent talks to a simulated user and calls tools until the episode ends, and the score checks the final state. There, the question a deployer needs answered is not only whether the average moves, but which task consequences quantization causes: when the quantized agent does something different, does that difference break the task, fix it, or wash out?

Why aggregate success is not enough. Equal accuracy can hide many changed predictions (Dutta et al., 2024), and quantized τ^2-bench agents can keep a flat score while specific failure modes rise (Jang et al., 2026). In our own runs, W4 checkpoints change which tasks succeed by more than a numerics-only floor (Appendix D). A turnover count, however, does not say what caused each change. In a closed loop, once one action differs, the user's replies, the tool results and every later decision differ too, so a success difference between two independent trajectories mixes the effect of the first changed action with everything the agent does afterwards, and with run-to-run variation.

Why open-loop probes are not enough. Teacher-forcing the quantized model on reference contexts measures how its next-action distribution shifts, but not whether a shifted action matters once the loop continues. It can also mislead on its own terms: in our audit, an apparent "blurring" of act-versus-talk propensities conditional on the reference's choice was largely a selection effect that a sign-symmetric perturbation of the same size reproduces (Appendix A). Open-loop shifts, behavioural fingerprints and aggregate success are therefore background measurements in this paper, not answers.

What we build and run. We intervene on one decision at a time. Along the quantized agent's own trajectory, both the BF16 agent (B) and the quantized agent (Q) read the *identical* saved history; at the first position where their full actions differ semantically, we execute either action and continue with either agent to the end of the episode (Figure 1). The four branches separate three ques-

tions: does the current action matter (replace only it), does the subsequent execution matter (replace only the continuation controller), and do they interact? Our contributions are the evaluation we build and execute:

1. **Run conditions and measurement definitions** for quantized closed-loop agents: byte-deterministic serving with a numerics canary, named success endpoints per domain, a tool-type table verified against the benchmark source, and reference controls kept separate from the quantization comparison (§3).

2. **A same-state four-branch design** with explicit estimands: the one-replacement gain G as rescues minus new damage, the decomposition of the total change into current-action, continuation and interaction terms, a full-pool primary endpoint in which tasks without a divergence contribute zero, replay acceptance criteria, and paired-block inference over independent nonce blocks (§4).

3. **A consequence and mechanism annotation protocol** that judges defects from tool preconditions, task requirements, authorisation and state changes rather than from disagreement with BF16, with blinded human review and reported confusion matrices (§5).

4. **An independent confirmation** on a telecom split held out from all development, with a frozen protocol and pre-declared thresholds (§6), and a transparent record of retired lines, including a pre-registered non-replication and a corrected statistical test (Appendices A–C).

We run the design on Qwen3-4B and Qwen3-8B with GPTQ-W4 and AWQ-W4 in the τ^2-bench retail, airline and telecom domains. [PENDING: one-paragraph summary of what the development forks and the holdout confirmation show; from §4.5 and §6 only]

2 Related Work

Same score, different behaviour. Dutta et al. (2024) show that compressed models with matched accuracy flip up to 13.6% of answers and argue for flip-based evaluation; Singh (2026) find per-item churn about $5\times$ the net change and propose paired equivalence tests at a declared margin. Both work on static benchmarks and count changed answers. We work in a closed loop, where a changed action is not an answer but an intervention whose effect depends on what happens next, and we ask what each change causes rather than how many changes occur.

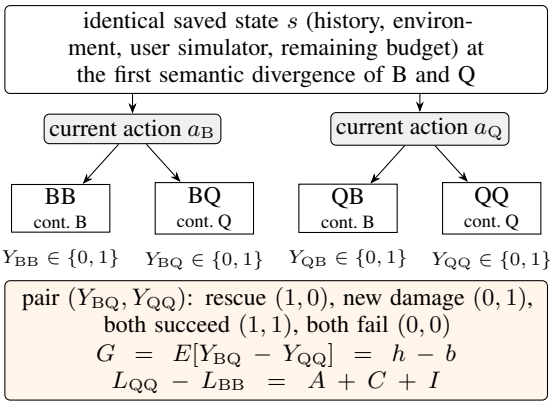


Figure 1: Schematic of one fork state (not data). B = BF16 agent, Q = W4 agent. A branch XY executes the current action a_X and then continues with controller Y to the end of the episode. The outcome categories and estimands are defined in §4; no numbers are shown because none are drawn from runs.

Quantized agents. *Flat Score, Amplified Failures* (Jang et al., 2026) is the closest study: BF16, FP8 and AWQ-INT4 agents on τ^2-bench retail and telecom with four trials per task, no aggregate change surviving correction, TOST at ± 7.5 points, tool-name hallucinations up to $2.5\times$, and a tighter error budget re-exposing a gap. It compares independently run trajectories, so a failure cannot be traced to the quantized action that started it or separated from what the agent did afterwards; we fork identical histories to make that separation. Benchmarks of compressed agents (Dong et al., 2025) and studies of task-dependent precision needs (Lu et al., 2026; Zhang et al., 2026) report scores, not consequences of individual actions. *Dynamic Mixed-Precision Routing* (Li et al., 2026) reports that interaction steps differ in their sensitivity to precision and learns step-level routing between a high- and a low-precision model. Our one-replacement gain G is the quantity such a router would harvest by sending one decision to the reference model; we measure it causally at the state where the two models first disagree, rather than learning a proxy for it.

Detecting that a model changed. Model Equality Testing (Gao et al., 2025) detects, from samples, whether an API serves the claimed model; AgentProv (Wang et al., 2026) audits agentic API providers with tool-use policy probes and detects

silent quantization. Detectability of a difference is a different property from its consequence: a quantized agent can be perfectly distinguishable and still cause no task loss, or cause loss through a handful of actions. We take the difference as given and measure its consequences.

Forking agent trajectories. *The Replay Gap* (Gonuguntla, 2026) evaluates model switching in SWE agents by forking live trajectories, with same-model control forks as a reference; a model swap rewrites most post-fork actions, and control-fork divergence depends on the serving format (FP8 controls diverge in over 90% of cases, AWQ controls stay near-identical). We share the forking idea but differ in the intervention and the estimand: we fork at the first quantization-induced action divergence, swap the current action and the continuation controller separately, and score the task outcome, so the change in success splits into current-action, continuation and interaction terms. Our serving is byte-deterministic (§3), so a same-checkpoint fork is required to reproduce its original exactly; it is an acceptance test (§4.3) rather than a noise reference.

Noise in agent evaluation. Agent scores vary across identical-setting runs: on τ^2-bench, single runs must differ by 7–8 (retail) or 10–11 (airline) points to exceed noise under sampling (Universeyi, 2026); pass@1 moves 2.2–6.0 points at temperature 0 on SWE-bench Verified (Bjarnason et al., 2026); inconsistency across repeated trials has been quantified with intraclass correlation (Mustahsan et al., 2025), and cost-controlled, reproducible agent benchmarking has been advocated (Kapoor et al., 2024); prompt perturbations inflate paired variance far beyond reruns on single-turn function calling (Chen et al., 2026); and benchmark construction itself can create false successes (Zhu et al., 2025). Serving numerics are a known source of greedy-decoding divergence (Du et al., 2026; Yuan et al., 2025; He and Thinking Machines Lab, 2025). These works motivate our choices to remove serving noise, to treat prompt nonces as a researcher-defined perturbation whose independent draws form replicate blocks, and to keep conclusions about a fixed task pool separate from conclusions about new tasks.

3 Run Conditions and Measurement Definitions

Tasks. We use τ^2-bench (Barres et al., 2025) in text mode. *Development* uses the 85 retail and airline tasks of our frozen evaluation split (58 retail, 27 airline; 22 tasks were excluded earlier because BF16 was unstable on them, and they stay out because that selection used BF16 behaviour) and the 80-task telecom development split. *Confirmation* uses the remaining 34 telecom tasks. The telecom split was fixed before any run from τ^2-bench's official 114-task `base` split, stratified by issue type and persona and ordered by a salted hash of the task id, so no model output entered it. A disjoint set of 57 retail/airline tasks supplies the quantizers' calibration trajectories.

Agents and quantization. The agents are Qwen3-4B-Instruct-2507 (q4) and Qwen3-8B (q8, thinking disabled), pinned to fixed revisions and served with the Hermes tool parser. The user simulator is Qwen3-30B-A3B, shared by all agents. Q is a GPTQ-W4 (Frantar et al., 2023) or AWQ-W4 (Lin et al., 2024) checkpoint, fake-quantized: weights are quantized and dequantized, then served with BF16 compute on the same serving path as the BF16 reference B. The primary checkpoint per model and method (calibration draw d1) was fixed by a completeness rule stated before any fork outcome (the checkpoint with the most existing independent runs; ties broken by the lowest draw index), not by earlier results. Agent and simulator decode greedily with a 32k-token context.

Deterministic serving. Agents and simulator run on vLLM with batch-invariant kernels and deterministic Inductor compilation; every server start checks a numerics canary against stored reference outputs. We adopted the deterministic compilation flag after finding that compile-time autotuning chose different reduction configurations for a fused residual+RMSNorm kernel on different nodes, which shifted every token of a determinism pair (Appendix E). With the fix, cross-node determinism pairs are byte-identical in prompts and completions: 257/257 decisions for q4 and 303/303 for Qwen3-30B-A3B as agent. Each branch of §4 is therefore designed to have one reproducible outcome given its state; the replay acceptance gate (§4.3) checks this rather than assuming it.

Prompt nonces and blocks. A *nonce* is a short random string inserted as a session-identifier line in the system prompt. It is a researcher-defined input perturbation, not a "no-change" null: it changes the input and re-rolls trajectories. We use it for one purpose: each independent nonce defines a *block*, one closed-loop run of Q (and of B, for paired success) over the task pool, and blocks are the replicate unit of inference (§4.4). Reference controls are reported separately and never pooled with the quantization comparison: identical-setting repeats, nonce perturbation, numerics controls (a rotated-BF16 checkpoint) and fixed random weight perturbations, the last being a control intervention rather than a null. [PENDING: stratified floor table for these controls, flagging single canonical-run floors — identical-setting repeats and a BF16-vs-BF16 nonce control are not run yet (`docs/insights/METHODOLOGY_REVISION.md` B4)]

Success endpoints. For retail and airline we report *DB success*: the final database matches the gold database and the episode terminated normally. This is not τ^2-bench's official reward, which multiplies DB match with natural-language assertions (retail, 112 of 114 tasks) or a communicate check (airline, 50 of 50); official rewards are reported separately. For telecom we report the official deterministic task-basis success (environment assertions, plus action checks for 14 of the 80 development tasks). Context overflow and unparseable agent output count as failures; infrastructure outages are missing, never failures, and never "no divergence".

Tool types. Whether an action reads, writes or hands off is taken from a table derived from τ^2-bench's tool decorators and re-verified against the source (Table 1); names outside a domain's tool list are *invalid*. An earlier name-pattern rule agreed with the table on 99.98% of q4/q8 episodes for "any write" in retail/airline, but misses 5 of the 6 telecom write tools, so the table is binding for all annotation and is frozen by hash before confirmation.

Background measurement: turnover above a floor. As motivation only, and only for the conditions where it was verified (5 Qwen3 models, retail and airline, W4 cells against a numerics floor of rotated BF16, canonical BF16 reruns and Gaussian noise at $k \leq 0.003$), the turnover intensity of

Domain	read	write	handoff	other	regex misses
Retail	7	7	1	1 (calculate)	0
Airline	6	6	1	1 (calculate)	0
Telecom	6	6	1	0	5 of 6 writes

Table 1: Agent tool types per domain, from the τ^2-bench tool decorators (commit `2174a60`), verified by re-derivation from the source. "Regex misses": write tools the earlier name-pattern rule failed to classify. Source: `analysis/insights/iter_4/out/tool_types.json`; full list in Appendix F.

W4 checkpoints exceeds the floor: pooled excess $+0.19$ [0.10, 0.30] for RTN, $+0.15$ for GPTQ and $+0.16$ for AWQ, Holm-adjusted (Appendix D). This says that W4 changes which tasks succeed. It does not say which of those changes quantized actions cause, which is the question of §4. Whether W4 success is non-inferior to BF16 is a separate, paired analysis (margin 3 pp; §6). [PENDING: paired Q−B success non-inferiority, TOST and ordinary difference per model × method — `analysis/insights/iter_4/success_ni.py` real mode on the `a6k2_{q4,q8}_bfpair_q1..q5` runs]

4 Same-State Four-Branch Design

4.1 Fork states: the first semantic action divergence

For a model, a quantized checkpoint Q, a task i and a block r, take Q's closed-loop trajectory. Before each of Q's decisions t we restore the saved state s_t: the full message history (agent, user and tool messages), the environment and user-device databases rebuilt by strict replay of the recorded tool calls, the user simulator's visible conversation, and the step and error budget already used. B then generates its full action on the same request; the prompt token ids B's server receives must equal those Q received at t, otherwise the task is excluded with that reason. The primary state is the first t at which the two full actions differ *semantically*: in class (tool call, text, stop), in tool name (including names outside the domain's tool list), in the number or order of calls, or in arguments after normalisation (key order, JSON whitespace and number formatting ignored).

- At most one state per task × model × checkpoint × block, frozen (written to disk with its hashes) before any branch is run.
- Differences in free text only (same class, differ-

ent wording) are recorded but are not primary states. Such differences, for example a commitment made or an authorisation requested in prose, can matter; they are outside the primary scope.

- Late divergences are kept, with their remaining step and error budget.
- A task with no qualifying divergence stays in the denominator with zero effect. Infrastructure failures are never coded as "no divergence".

4.2 Branches and estimands

From state s, the current action a_{X} ($\mathrm{X} \in \{\mathrm{B}, \mathrm{Q}\}$) is executed as the agent's next step by the real environment or answered by the real simulator, consuming one step and counting its errors as in live play; controller Y then continues to the end with the original remaining budget. This gives branches BB, BQ, QB and QQ with binary outcomes Y_{XY} and means $L_{\mathrm{XY}} = E[Y_{\mathrm{XY}}]$ over fork states (Figure 1). QQ reproduces Q's original episode.

One-replacement gain. Replacing only the current action with B's, while Q continues, gives

$$G = E[Y_{\mathrm{BQ}} - Y_{\mathrm{QQ}}] = h - b, \qquad (1)$$

with rescue rate $h = P(Y_{\mathrm{BQ}} = 1, Y_{\mathrm{QQ}} = 0)$ and new-damage rate $b = P(Y_{\mathrm{BQ}} = 0, Y_{\mathrm{QQ}} = 1)$. We always report the paired four-way composition (rescue, new damage, both succeed, both fail), because the same G can come from few consequences or from many rescues cancelled by many breaks.

Decomposition. With B's branch BB as the reference,

$$A = L_{\mathrm{QB}} - L_{\mathrm{BB}} \quad \text{(current action)}, \qquad (2)$$
$$C = L_{\mathrm{BQ}} - L_{\mathrm{BB}} \quad \text{(continuation)}, \qquad (3)$$
$$I = L_{\mathrm{QQ}} - L_{\mathrm{QB}} - L_{\mathrm{BQ}} + L_{\mathrm{BB}} \quad \text{(interaction)}, \qquad (4)$$

so that $L_{\mathrm{QQ}} - L_{\mathrm{BB}} = A + C + I$ exactly, and $G = -(A + I)$: the gain of one replacement is the current-action effect as seen under Q's own continuation. A, C and I are contrasts on the population of fork states; they say where the change in success at a divergence comes from, not whether a particular action was wrong (§5).

Primary endpoint. Let $D_{ir} = 1$ if task i in block r reaches a valid fork state. The primary endpoint is the full-pool one-replacement gain

$$G_{\mathrm{all}} = E_{i,r}\left[D_{ir}\,(Y_{\mathrm{BQ}} - Y_{\mathrm{QQ}})_{ir}\right] = \pi\, G, \qquad (5)$$

with divergence prevalence $\pi = P(D_{ir} = 1)$, averaged over the whole task pool with fixed cell weights declared in advance; tasks without a divergence contribute zero. The conditional G (given a divergence) is secondary and is estimated as a ratio with block resampling. Prevalence, the number of valid states and every exclusion reason are always reported.

4.3 Replay acceptance

Forks count as mechanism evidence only if an engineering gate passes before any development outcome is analysed: (a) restoration of the databases, device state, user-simulator state and step budget is correct, and canonical state hashes match after save and load; (b) no-op forks B→B and Q→Q reproduce the original trajectories and outcomes, token for token; (c) with B and Q pointed at the same checkpoint (on two servers), all four branches agree; (d) branches are isolated, so execution order and concurrency do not change any branch; (e) after a controller switch the context is re-prefilled from the history by the other server, with no KV or prefix cache shared across weights; and forced actions consume the same step and error budget as live actions. If restoration cannot be verified by the decision point, the paper reports the measurements of §4.1 and §5 and withdraws the causal reading of the branches. [PENDING: gate outcome (a)–(e) with counts — `analysis/fork/gate/` (`gate.json`, `gate_same.json`) from `scripts/a6000/fork_driver.py`; `docs/insights/FORK_PROTOCOL.md` §8]

4.4 Protocol, replicates and inference

Deterministic protocol. Under deterministic serving and a greedy simulator, each branch is run once per state; the randomness in G_{all} comes from which states arise, and that varies only across blocks. A block is one independent nonce (§3); all tasks of a block share it, and B and Q branches share each state, so inference is on paired blocks: we compute each block's full-pool statistics and draw intervals from between-block variation, with simulation checks of calibration for sparse events. [PENDING: interval method and

its simulated coverage for sparse rescue/damage counts — statistics deliverable (review §D), script in `analysis/fork/` not yet written]

Stochastic fallback. If deterministic replay fails the gate but the start state (environment, history, remaining budget) is verifiably identical across branches and the remaining differences come from a describable, repeatable source of randomness in the subsequent execution, the estimand becomes $G = E[Y_{\mathrm{BQ}}] - E[Y_{\mathrm{QQ}}]$, the expectation over that declared randomness. Branch repeats estimate these expectations and are nested within task, nonce and state; they are not independent samples. Same-configuration replays measure run variance but are never subtracted from G as a "no-op flip rate". Hidden-state loss, branch contamination or systematic replay bias cannot be repaired by repeats. Repeat count, seed rule, failure handling and inference would be fixed in the protocol before use.

Two target populations. Conclusions about the *fixed benchmark task pool* treat tasks as fixed and blocks as random; conclusions about a *new task distribution* resample tasks and blocks jointly (task-clustered, crossed). More nonce blocks narrow the first but not the second; only more independent tasks narrow the second. We report both and never let the first stand in for the second.

Decision rules. We plan power for a true G of $g_{\min} = 3$ pp. G is called positive if its one-sided 95% lower bound exceeds 0, and above 3 pp if that bound exceeds 3 pp. An average effect is called small only if a valid 90% interval lies inside $(-3, +3)$ pp. A 5 pp margin is a sensitivity analysis only. A pooled pass does not imply a per-model or per-method pass. [PENDING: multiplicity rule across models, methods and secondary endpoints — to be frozen in `docs/insights/PREREG_FORK_CONFIRM.md`]

4.5 Development results

Development covers q4 and q8 × GPTQ-W4 and AWQ-W4 on the 85 retail/airline tasks and the 80 telecom development tasks. The number of blocks is set by simulator throughput, after the gate, full task coverage and complete, balanced blocks, with capacity reserved for the confirmation (Table 2).

5 Consequence Composition and Mechanism Annotation

A different full action is not by itself a mechanism finding, and a nonzero G does not say why. For each fork state and branch we therefore record process evidence in three independent fields. The BF16 action is only a reference: a Q action is never labelled defective because it differs from a_{B}, and a_{Q} and a_{B} are judged by the same checks.

1. **Defect of the current action**, judged from tool preconditions, task requirements, authorisation and state changes: an invalid tool (Table 1), invalid arguments against the tool schema, an execution error returned by the environment, identifiers that appear in no earlier message or tool output, a write without an explicit confirmation in the preceding user turn (a proxy, not an authorisation verdict), a malformed action, or ending the episode.
2. **Later correction aimed at that defect**, e.g. a later valid call of the same tool with grounded identifiers; a write is not undone by a retry.
3. **Requirements satisfied at the end**: success under its named endpoint, each environment assertion, action and communicate check, and the database paths that differ from the gold state.

Recovery is claimed only for the sequence *defect* → *correction* → *requirements satisfied*. Otherwise a failed branch reads "not recovered under the tested continuation controllers and the fixed remaining budget". We also relate non-recovery to the remaining budget, the defect type and the controller.

Three layers of labels. (i) *Automatic events* are extracted on all trajectories from the tool-type table, the tool schemas, environment responses and the evaluator, keeping the raw record behind each judgement. (ii) An *LLM labeller* assigns correction and failed-requirement labels under fixed written rules; every label carries its evidence location in the trajectory, and "cannot judge" is an allowed answer. Labels such as "wrong identifier", "write without consent" and "premature end" need rule support or human review, because legitimate substitutes, earlier authorisation and already completed tasks exist. (iii) *Blinded human review* covers about 80 fork states, stratified by consequential versus non-consequential pair, tool type and automatic label; about 20 are reviewed independently by two people. Blinding strips model, precision

	Quantity	q4 GPTQ-W4	q4 AWQ-W4	q8 GPTQ-W4	q8 AWQ-W4	Pooled (fixed weights)
(a) Divergence coverage	tasks in pool × blocks	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
	valid fork states	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
	no qualifying divergence	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
	excluded (by reason)	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
	prevalence π [CI]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
	position t / remaining budget (median)	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]	[PENDING: DEV-a]
(b) Consequence composition $(Y_{\mathrm{BQ}}, Y_{\mathrm{QQ}})$	rescue $(1,0)$: n (share)	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]
	new damage $(0,1)$: n (share)	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]
	both succeed $(1,1)$: n (share)	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]
	both fail $(0,0)$: n (share)	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]	[PENDING: DEV-b]
(c) Gain and decomposition (pp, 95% CI)	G_{all} (primary, fixed pool)	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]
	G_{all} (new tasks)	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]
	G given divergence	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]
	A (current action)	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]
	C (continuation)	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]
	I (interaction)	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]	[PENDING: DEV-c]

Table 2: Development forks (85 retail/airline + 80 telecom development tasks), per model × method and pooled with fixed cell weights. All cells are placeholders. [PENDING: DEV-a: coverage from the frozen state files (`states/*.json`) written by `scripts/a6000/fork_driver.py`; DEV-b, DEV-c: branch outcomes from `fork_driver.py` and the paired-block analysis script to be added under `analysis/fork/`; per-domain split in the appendix]

Quantity	q4 GPTQ-W4	q4 AWQ-W4	q8 GPTQ-W4	q8 AWQ-W4	Pooled (fixed weights)
blocks / valid fork states / no divergence / excluded	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]
rescue / new damage / both succeed / both fail: n (share)	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]
G_{all} (pp) [one-sided 95% lower bound]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]
A, C, I (pp) [95% CI]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]	[PENDING: HOLD]
frozen hypothesis: verdict			[PENDING: HOLD]		

Table 3: Telecom holdout confirmation (34 tasks). All cells are placeholders. [PENDING: HOLD: holdout fork runs under the frozen `docs/insights/PREREG_FORK_CONFIRM.md` (not yet written), analysed by the same `analysis/fork/` script as Table 2]

and branch source and randomises order. We report stratified confusion matrices of human against automatic and LLM labels for the key defect and recovery labels; because the sample is enriched, these do not estimate whole-data accuracy. Case studies are drawn only from human-verified states.

Reading G near zero. The composition and the process fields decide how a small G is read: a wide interval is *inconclusive*; many rescues and many breaks mean *cancellation*; branches that usually both succeed mean a *small consequence*; and an explicit Q defect, then a correction, then success is *observed recovery*.

Development evidence gate. There is no go/no-go on the sign of G. Development ends when divergence consequences are measured clearly enough to yield a specific explanation that the holdout can test: stable consequences with mechanism evidence; a small effect with a narrow interval and a checked rescue/damage balance; or a few pre-defined divergence types with their opportunity denominators. If intervals are wide and events few, we report insufficient evidence and no new explanation. [PENDING: which of these outcomes development produced, and the frozen hypothesis for §6]

Defect type (current Q action)	states	new damage	recovered
invalid tool / invalid arguments	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
execution error	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
ungrounded identifier	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
unconfirmed write	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
ends episode	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
no verifiable defect	[PENDING: MECH]	[PENDING: MECH]	[PENDING: MECH]
human vs. auto / LLM agreement		[PENDING: MECH-h]	
double-review agreement (≈20)		[PENDING: MECH-h]	

Table 4: Process evidence for development fork states. "Recovered" requires defect → correction → requirements satisfied. [PENDING: MECH: `annotations.jsonl` from `cloq/fork_annotate.py` plus LLM labels (labeller and rules not yet fixed); MECH-h: blinded review sheets and confusion matrices, stratified, in the appendix]

6 Independent Confirmation

Holdout. The 34 telecom holdout tasks have not been run by any agent in this study. Telecom differs from retail and airline in its tools and requirements, and in dual control: the user acts on a simulated device, so both sides' tool calls change state. Before the holdout is opened, the mechanism categories found in development are mapped to telecom tools and requirements; the telecom development tasks may be used to debug the tool ta-

ble and to check that the categories fit, with every rule change logged. The protocol, the tool-type table (by hash), the hypothesis, the number of blocks and the stopping rule are then frozen in a pre-registration file. The block count is derived from the development block variance, not fixed in advance.

Thresholds and their rationale. The confirmation uses the thresholds of §4.4: $g_{\min} = 3$ pp for G, and, for the background success comparison, a non-inferiority margin $\varepsilon = 3$ pp (H_0: $\text{success}_Q - \text{success}_B \leq -\varepsilon$; pass if the one-sided 95% lower bound exceeds -3 pp). The margin was written down before any paired BF16 outcome existed. On the 85-task pool, 3 pp is a change of about 2.5 task outcomes per run. The run-level nonce SD of one checkpoint was 2.5 pp on q4 and 2.9 pp on q8, so a smaller margin could not be reached with a few paired blocks; a 5 pp margin would sit at the size of the largest W4 checkpoint effect we observed (q8 AWQ-W4 d3, $+5.3$ pp on average), so it could not flag such a checkpoint. Using one margin for G and for success keeps "negligible" consistent. Non-inferiority is not equivalence: equivalence additionally needs the two one-sided tests, with the 90% interval inside $(-3, +3)$ pp, and a margin the data cannot resolve is reported as inconclusive.

What counts as confirmation. The frozen development hypothesis is tested on the holdout with the same estimands and annotation. With 34 tasks, few holdout events for a mechanism mean limited evidence for it, not its absence; and fixed-pool conclusions on the holdout do not by themselves generalise to new tasks (§4.4).

7 Discussion and Deployment Implications

[PENDING: written only after §4.5–§6 are filled. The discussion must name which of the pre-stated outcomes occurred — a stable gain; many rescues and many breaks with a small net effect; few consequences with a narrow interval; verified recovery; or damage concentrated in a few divergence types — and state what a deployer can conclude from it for the tested models, methods, simulator and task pool only.]

Independently of the results, the design has two practical readings. First, G_{all} is the expected benefit of sending exactly one decision per episode — the first one on which the quantized agent departs from the reference — back to the reference model; it therefore measures what step-level precision routing would gain at that one decision under the tested continuation, not over a whole episode. Second, the decomposition tells an evaluator where to look: a large C with a small A points to the subsequent execution rather than to any single action, whereas a large A points to the actions themselves and to the annotation of §5. Which reading applies here is an empirical question that the tables above will answer.

Limitations

Quantization and serving. Q checkpoints are fake-quantized: dequantized W4 weights with BF16 compute on a batch-invariant serving path. Deployed low-bit kernels change the numerics and would add serving variation that we deliberately remove, so our results describe the effect of the weight change, not of a production kernel. We study two Qwen3 models and two W4 methods, one primary checkpoint each; checkpoint-to-checkpoint variation within a method is not covered by the fork results.

One simulator, one benchmark. All episodes use one τ^2-bench user simulator (Qwen3-30B-A3B, greedy) and three τ^2-bench domains. Continuation effects include the simulator's responses, so a different simulator may change C and I. Retail and airline are scored by DB success, which is not τ^2-bench's official reward; official rewards are reported separately, and telecom uses its official deterministic task-basis success.

Blocks from a researcher-defined perturbation. Replicate blocks come from prompt nonces, an input perturbation we chose; they are not draws of a natural deployment distribution. Conclusions for the fixed task pool and for new tasks are reported separately. In a development run of the paired success analysis with stand-in pairs (not a result), the task-clustered interval for new tasks had a half-width of about ± 4 pp for the all-model pool, so claims at a 3 pp resolution about new tasks are unlikely with the current task counts; more independent tasks, not more nonces, would narrow it.

Scope of the fork. Only the first semantic action divergence per task and block is intervened on; later divergences, and differences in free text only

(which can carry commitments or authorisation requests), are outside the primary scope. The causal reading of the branches depends on the replay acceptance gate; if restoration cannot be verified, the paper reports measurements only. Mechanism labels depend on the tool-type table, on rule-based checks and on an LLM labeller; about 80 human-reviewed states validate them on an enriched sample, which does not give whole-data accuracy.

Small confirmation set. The telecom holdout has 34 tasks. Few events there limit the evidence for any mechanism and do not show its absence; non-inferiority at 3 pp is not equivalence.

Retired lines, reported transparently. This project previously pursued a "checkpoint lottery" (same method, different calibration draw, different success) and an account in which W4 agents matched BF16 success while behaving differently. The first was tested in a pre-registered confirmation and the exploratory results did not replicate; this is not evidence that checkpoint effects are absent, nor of BF16/W4 success equivalence (Appendix B). For the second, success equivalence was never established, and the pre-registered fingerprint test turned out to have an invalid permutation null; after correction, no fingerprint was detected for W4 quantizer checkpoints (Appendices A, C). Both are kept in the appendix as background, and neither supports a claim in the main text.

Ethics Statement

This work involves no crowdworkers and no personal data: all users are simulated by an LLM on the public τ^2-bench tasks, and all agent models are openly released Qwen3 checkpoints. The planned human review of about 80 fork states (§5) concerns synthetic benchmark conversations only. [PENDING: who the reviewers are and their instructions — to be fixed with the annotation guidelines] The main risk we see is over-reading: a small or cancelling one-replacement gain on one simulator and a fixed task pool does not certify a quantized agent for deployment, and non-inferiority at a 3-point margin is not equivalence. Deployment decisions in consequential settings should not rest on these measurements alone. Compute use is reported in the Responsible NLP checklist.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. 2026. On randomness in agentic evals. *arXiv preprint arXiv:2602.07150*.

Yihang Chen, Pin Qian, Su Wang, Chong Peng, Huan Xu, Xiyang Wu, and Yiqi Sun. 2026. Noise floor audit for agent benchmarks. *arXiv preprint arXiv:2608.22331*.

Peijie Dong, Zhenheng Tang, Xiang Liu, Lujun Li, Xiaowen Chu, and Bo Li. 2025. Can compressed LLMs truly act? an empirical evaluation of agentic capabilities in LLM compression. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Gaoyuan Du, Anam Nawaz Khan, Rex Zhou, Xiaoyang Liu, Deepayan Chakrabarti, Fnu Suya, and Xueping Li. 2026. Greedy decoding is not Precision-Invariant: Cross-Precision output divergence in LLM inference. *Transactions on Machine Learning Research*.

Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is not all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*.

Irena Gao, Percy Liang, and Carlos Guestrin. 2025. Model equality testing: Which model is this API serving? In *ICLR*.

Ashritha Gonuguntla. 2026. The replay gap: Static evaluation of model switching in LLM agents scores the wrong world. In *Conference on Language Modeling (COLM)*.

Horace He and Thinking Machines Lab. 2025. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism (blog). September 10, 2025.

Jiwon Jang, Kisu Yang, Heuiseok Lim, and Hyunwoo Park. 2026. Flat score, amplified failures: How the error budget masks damage in quantized LLM agents. *arXiv preprint arXiv:2607.27275*.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI agents that matter. *arXiv preprint arXiv:2407.01502*. Published in Transactions on Machine Learning Research according to Semantic Scholar.

Yuanzhe Li, Jianing Deng, Jingtong Hu, Tianlong Chen, Song Wang, and Huanrui Yang. 2026. Dynamic mixed-precision routing for efficient multi-step LLM interaction. *Preprint*, arXiv:2602.02711.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*.

Haiquan Lu, Zigeng Chen, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2026. Mix-Quant: quantized prefilling, precise decoding for agentic LLMs. *arXiv preprint arXiv:2605.20315*.

Zairah Mustahsan, Abel Lim, Megna Anand, Saahil Jain, and Bryan McCann. 2025. Stochasticity in agentic evaluations: Quantifying inconsistency with intraclass correlation. *Preprint*, arXiv:2512.06710.

Amogh Singh. 2026. Certifying compressed language models: An audit and a statistical toolkit. *arXiv preprint arXiv:2608.15046*.

Universeyi. 2026. Run-to-run noise floor of the published baselines and leaderboard submissions, computed from the released trajectory files. GitHub issue #540, sierra-research/tau2-bench. Opened 2026-09-12.

Xun Wang, Bihe Zhao, Michael Backes, Franziska Boenisch, and Adam Dziedzic. 2026. Agentprov: Auditing agentic llm api providers via tool-use policy probes. *Preprint*, arXiv:2609.00052. EMNLP 2026.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Manyi Zhang, Ji-Fu Li, Zhongao Sun, Xiaohao Liu, Zhenhua Dong, Xianzhi Yu, Haoli Bai, and Xiaobo Xia. 2026. QuantClaw: precision where it matters for OpenClaw. *arXiv preprint arXiv:2604.22577*.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, and 6 others. 2025. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*.

A P0 Audit: Statistics and Measurement Fixes

An external review of an earlier version of this project identified statistics and measurement problems; all were confirmed in the code and fixed before the fork study. Everything below that touches the 28 pre-registered confirmatory runs (Appendix B) is a *corrected, post-hoc* re-analysis; the pre-registered outputs are kept unchanged as history. The re-analysis reproduces the pre-registered numbers exactly before applying fixes (H3 $F = 0.2602$, $p = 0.0008$; H1 $p = 0.74385$).

H3 correction (split-task fingerprint). The pre-registered test scaled each behaviour feature by the differences of the *true* checkpoint pairs and then permuted checkpoint labels of a cosine matrix computed with that scale held fixed. The scale depends on the observed pairing, so permuted statistics are not exchangeable with the observed one. The corrected (main) test uses a pairing-invariant, unpaired pooled scale per split and feature, which makes the column permutation a valid permutation test; a sensitivity test recomputes the original preprocessing inside every permutation. Features, the 100 split halves and relabelling streams are fixed across variants.

Family (F / one-sided p)	relabel.	original (invalid)	main (corrected)	sensitivity
Pooled, 4 blocks (= H3)	5000 MC	0.260 / 0.0008	0.136 / 0.0134	0.260 / 0.0028
RECAL only	576	0.118 / 0.0052	0.025 / 0.304	0.118 / 0.130
RAND only	36	0.450 / 0.028	0.283 / 0.028	0.450 / 0.028
q4	144	0.193 / 0.021	0.088 / 0.125	0.193 / 0.069
q8	144	0.328 / 0.0069	0.184 / 0.049	0.328 / 0.021
within model×method, 6 blocks	576	0.260 / 0.0017	0.158 / 0.0052	0.260 / 0.0035
within, GPTQ/AWQ blocks only	16	0.117 / 0.0625	0.064 / 0.1875	0.117 / 0.25

Table 5: H3 before and after correction. RECAL = W4 quantizer recalibrations (GPTQ/AWQ d3, d4); RAND = fixed Gaussian checkpoints ($k = 1$, seeds s2, s4, s5). Exact enumeration where relabellings ≤ 5000; RAND's $p = 0.028$ is the minimum attainable $(1/36)$. RAND carries 74% of the original pooled F and 90% of the corrected one. Source: `analysis/insights/iter_4/out/h3_corrected_20260930T0044Z.json`; `docs/insights/METHODOLOGY_REVISION.md` B1.

Calibration by simulation. Synthetic datasets resample the real per-task shift rows within block, so draws are exchangeable. At $\alpha = 0.05$ the original test rejects 0.364 ± 0.011 (20 splits, $B = 199$, 2000 datasets), 0.331 ± 0.006 (independent seeds, $B = 99$, 6000 datasets) and 0.417 ± 0.011 (100 splits, $B = 499$) of null datasets; the corrected test rejects 0.053 ± 0.005, 0.044 ± 0.003 and 0.061 ± 0.005 against nominal levels of at most 0.045, 0.040 and 0.048. The 100-split rows share datasets and sit 1.9–2.4 SE above nominal; we read them as a possible excess of at most 0.01. On i.i.d. synthetic shifts, the corrected test rejects 0.0405 ± 0.0025 (nominal ≤ 0.040), the original 0.81. Power of the corrected test at planted signals $\delta =$

$0.01/0.02/0.03/0.05$: 0.07/0.12/0.24/0.75. The pre-registered "H3 supported" label is removed; the corrected pooled result is post-hoc and is carried by the random checkpoints (Appendix C).

Crossed variance components. The pre-registered overdispersion estimator subtracted a nonce term that included the shared nonce main effect, biasing ϕ low. Replacing it by a crossed model $m_{jn} = \mu + a_j + b_n + e_{jn}$ leaves the H1 permutation p unchanged (the removed term is invariant under checkpoint relabelling): pooled RECAL ϕ is 0.396 (pre-registered), 0.469 (main effect removed; $p = 0.744$, identical) and 0.327 (fully crossed; $p = 0.721$). The H1 verdict is unchanged. Crossed components over GPTQ/AWQ-W4 d1–d4 (Henderson method III; signed SD in pp, $\text{sign}(v)\sqrt{|v|}$):

Model	residual (nonce × ckpt + run)	checkpoint	nonce main	method
q4 GPTQ+AWQ	2.62	3.69	−0.86	−2.07
q8 GPTQ+AWQ	2.61	1.24	1.26	0.30

Table 6: Crossed variance components of run-level success. Method has 1 df and is not estimable; a negative value is a moment estimate, not evidence that a component is zero. q4's checkpoint component is driven by d1/d2; d3/d4 are homogeneous. Source: `analysis/insights/iter_4/out/p1_crossed_20260930T0044Z.json`.

Measurement fixes.

- *Open-loop act probability.* A tool-start token outside the stored top-20 was counted as probability 0. Now reported with lower/upper bounds and a missing flag; means move by less than 0.001, although the flag is frequent (17–34% of q4 positions for W4 and Gaussian cells).
- *Top-k KL.* Renamed to an approximation; an exact coarse-grained KL over tokens known on both sides plus one residual category (a lower bound) is added. Rankings are unchanged.
- *Tool-name mass.* The prefix-tree estimate is demoted to a proxy with measured error terms; the main tool metric is the fully generated and parsed name and arguments, as in the forks.
- *Confirmation feature.* A regex on the agent's own text was named as if it measured authorisation; it is renamed a confirmation-request proxy. Real authorisation needs user replies, tool rules and state, as in §5.
- *Write classification.* Replaced by the verified tool-type table (Table 1); 1.6% of q4/q8 episodes contain an invalid tool call.
- *Divergence analysis.* The earlier first-divergence analysis compared two independent trajectories, which stop sharing a history after the first difference; it cannot serve as a same-state fork. The fork design replaces it.
- *Success naming.* "Success" for retail/airline was the DB end-state match; it is now named DB success, with official rewards reported separately (§3).

Act/talk selection effect. Conditioning the quantized model's act probability on the reference's own choice selects positions where the reference's probability is near 0 or 1, so any perturbation, even a sign-symmetric one, pulls both conditional means toward the middle. For q4 RTN-W4, $P(\text{act} \mid \text{BF16 talked})$ rises from 0.014 to 0.088, while a sign-symmetric perturbation of the cell's own logit shifts predicts 0.080. The remaining directional part is a small net shift toward talking (unconditional $\Delta P(\text{act})$ between -0.10 and $+0.005$ on q4/q8 W4 cells), and size-matched random perturbations move it as much or more. Source: `analysis/insights/iter_4/out/act_talk_selection_20260930T0044Z.json`.

B Iteration-4 Pre-Registration Record

Hypotheses H1–H4 were registered before any confirmatory outcome was analysed, with 14 new checkpoints (RECAL: GPTQ/AWQ-W4 on new calibration draws d3, d4; RAND: Gaussian $k = 1$ seeds s2, s4, s5) × 2 fresh nonces on q4 and q8 (28 runs; an amendment before unblinding corrected a miscount of 40), 85 development tasks (q4: 84), Holm over H1–H3. The analysis ran once.

Hypothesis	estimate	p (Holm)	registered verdict
H1 recalibration lottery, $\phi_{\text{RECAL}} > 1$	0.40 (q4 −0.30, q8 0.94)	0.74	not supported
H2 net effect replicates across nonces, $r > 0$	0.42 (q4 −0.52, q8 0.94)	0.42	not supported
H3 fingerprint on disjoint task halves	F 0.26	0.0024	supported; *label removed after audit (App. A)*
H4 $\phi_{\text{RECAL}} - \phi_{\text{RAND}} \neq 0$ (secondary)	−0.83 [−7.07, 4.22]	0.70 (raw)	not detected (underpowered)

Table 7: Pre-registered confirmatory results. Source: `analysis/insights/iter_4/out/prereg_h1_h4_CONF_20260930T0044Z.json`; registration `docs/insights/PREREG_iter4.md` (sha256 prefix `a991e2e5ca2d6098`).

The exploratory checkpoint-lottery results that

motivated H1/H2 (a q4 GPTQ-W4 d1 versus d2 gap of +9.6 pp [1.8, 17.4]) did not replicate in the pre-registered confirmation. This is not evidence that checkpoint effects are absent, and not evidence of BF16/W4 success equivalence. The new q4 recalibrations landed at net +1.5 to +3.9 pp against the nonce reference; this range applies to q4 only. On q8, AWQ-W4 d3 was +5.3 pp on average (+8.2 pp under one nonce). Results are reported stratified by model and method; pooled averages support only their own weighted estimand.

C Random-Perturbation Fingerprint (Exploratory, Post-Hoc)

The corrected exploratory analysis detected cross-run behavioural consistency in random-perturbation checkpoints ($F = 0.283$, $p = 0.028$ for RAND only); no corresponding evidence was detected for W4 quantized checkpoints (RECAL only $F = 0.025$, $p = 0.30$; sensitivity $p = 0.13$; within-method $p = 0.19$). The RAND p is post-hoc, is the minimum attainable with 36 relabellings, and must be read with multiplicity. The pooled corrected result ($p = 0.013$) is 90% carried by RAND and supports no claim about quantization. With two recalibrations per method and model, a within-method test cannot reach $p < 0.5$.

D Turnover Above a Numerics Floor

Turnover λ is the intensity of a shrink-to-attractor model, $p'_i = (1 - \lambda)p_i + \lambda c$, fitted per cell against the model's nonce reference with misclassification correction; we keep it as an auxiliary model. The floor is the equal-weight mean of the available numerics-floor families (rotated BF16, canonical BF16 reruns, Gaussian $k \in \{0.001, 0.003\}$). Inference uses a hierarchical paired bootstrap over tasks, reference draws, checkpoints and draws, and Holm over a declared family of 14 pooled tests. Snapshot 20260929T1937Z, retail + airline.

E Determinism and Serving Details

The incident. In an earlier pass, a BF16 q4 determinism pair served on two nodes disagreed on 20/20 tasks and 113/113 identical-prompt decisions. The kernels were identical across nodes; torch Inductor chose the reduction configuration of the fused residual+RMSNorm kernel by timing it at compile time (a block size of 2048 on one node versus 4096 elsewhere), which changed the summation order and shifted every token. vLLM's batch invariance does not cover this. Setting `TORCHINDUCTOR_DETERMINISTIC=1` and checking a numerics canary (outputs compared by hash with a stored reference) for agent and simulator at every server start removed it: the re-run pairs were byte-identical in prompts and completions (257/257 decisions for q4, 303/303 for Qwen3-30B-A3B as agent). The whole matrix was re-run under the fix.

Method	q4	q8	q14†	q30a3†	q32†	pooled [95% CI]	p_{Holm}
RTN-W4	+0.17	+0.17	+0.18	+0.17	+0.29	+0.19 [0.10, 0.30]	0.007
GPTQ-W4	+0.22	+0.28	+0.08	+0.04	+0.13	+0.15 [0.07, 0.24]	0.007
AWQ-W4	+0.20	+0.31	−0.02	+0.13	+0.20	+0.16 [0.07, 0.25]	0.036

Table 8: Excess turnover $\lambda-$floor, one-sided. †Floor from canonical BF16 reruns only (one or two runs): a weak, single-family floor, flagged. Per model, RTN's interval excludes 0 only on q4 and q32. Source: `analysis/insights/iter_3/out/r10_turnover_stats_live_20260929T1937Z.json`; `docs/insights/iter_3_stats_R3.md`.

Configuration. vLLM 0.29 with `VLLM_BATCH_INVARIANT=1` and pinned compile flags; greedy decoding for agent and simulator; 32k-token prompt budget; two-GPU tensor parallelism only on NVLink-connected pairs. In the fork study, B and Q are served by separate server processes, so no KV or prefix cache is shared across weights. [PENDING: fork serving record: checkpoint hashes, canaries and prefix-cache counters per server — `logs/tasks/<forktask>.servers.json`]

F Tool-Type Table

Retail read: find_user_id_by_email, find_user_id_by_name_zip, get_item_details, get_order_details, get_product_details, get_user_details, list_all_product_types. write: cancel_pending_order, exchange_delivered_order_items, modify_pending_order_address, modify_pending_order_items, modify_pending_order_payment, modify_user_address, return_delivered_order_items. handoff: transfer_to_human_agents. other: calculate.

Airline read: get_flight_status, get_reservation_details, get_user_details, list_all_airports, search_direct_flight, search_onestop_flight. write: book_reservation, cancel_reservation, send_certificate, update_reservation_baggages, update_reservation_flights, update_reservation_passengers. handoff: transfer_to_human_agents. other: calculate.

Telecom read: get_bills_for_customer, get_customer_by_id, get_customer_by_name, get_customer_by_phone, get_data_usage, get_details_by_id. write: disable_roaming, enable_roaming, refuel_data, resume_line, send_payment_request, suspend_line (the earlier regex

caught only send_payment_request). handoff: transfer_to_human_agents.

G Per-Domain Development Results

[PENDING: Table 2 split by domain (retail, airline, telecom development) and by divergence category (class, tool name, call count/order, arguments), with remaining budget — same `analysis/fork/` script; stratified confusion matrices for the human review (§5)]